

Automated Eligibility Criteria Extraction

Technical Architecture, Outputs, Errors, and Development Roadmap

Modern AutoCrit Project

Contents

1 Purpose and Scope	2
2 Lineage and Changes from AutoCriteria	2
3 Current Web Application	2
3.1 User-facing workflow	2
3.2 Architecture	3
3.3 End-to-end extraction sequence	3
4 Function and Component Reference	4
4.1 Entry point, HTTP server, and state	4
4.2 Registry and trial parsing	4
4.3 Extraction jobs and persistence	4
4.4 LLM boundary and usage accounting	5
4.5 Segmentation, extraction, and EAV governance	5
4.6 Value normalization and clinical distance	6
5 Meaning of Application Outputs	7
5.1 Job output	7
5.2 Cleaned eligibility table	7
5.3 Clinical-distance output	8
6 Meaning of Errors	8
7 Persistence, Recovery, and Privacy	9
8 Key Milestones	10
9 Future Work	10
9.1 Fine-tuned embeddings for “seemingly similar” criteria	10
9.2 Broader roadmap	11
10 Conclusion	11

1 Purpose and Scope

Automated Eligibility Criteria Extraction is a local, browser-based application that converts free-text clinical-trial eligibility criteria into atomic, governed Entity–Attribute–Value (EAV) records. A user may supply either a ClinicalTrials.gov NCT identifier or pasted eligibility text. The application retrieves or accepts the protocol text, segments it into atomic criteria, uses a large language model (LLM) to map each criterion into the terminology library, normalizes categorical and numerical values, validates the mapping, and presents a reviewable table. It also exposes a clinical-distance calculator and a read-only view of the EAV library.

This report documents the current implementation rather than the retired desktop interface. It covers every active architectural layer and the public or workflow-defining functions in the project. Small private helpers are grouped with their parent subsystem where they implement one operation, such as interval comparison, canonical text cleanup, or HTTP serialization.

2 Lineage and Changes from AutoCriteria

The project was inspired by Datta’s AutoCriteria repository and paper [1, 3]. The inspiration demonstrated that an LLM could structure eligibility criteria. The current application is a substantial engineering extension:

- LangChain orchestration was removed in favor of direct OpenAI SDK access behind a small provider interface. This reduces framework coupling and makes prompts, schemas, token usage, and failures easier to inspect.
- The original long-script organization was decomposed into services, schemas, criteria-processing modules, statistical modules, and a browser presentation layer.
- The former standalone desktop GUI was replaced by a local web application served over HTTP. The browser is now the only supported interface.
- ClinicalTrials.gov API v2 retrieval and parsing allow an NCT identifier to serve as the protocol input.
- Extraction now includes atomic segmentation, a governed EAV terminology library, structured value and interval normalization, mapping review and reconciliation, job recovery, atomic persistence, and deterministic caching.
- A clinical-distance subsystem turns normalized numerical or categorical conditions into interpretable penalties for downstream statistical work.

These changes preserve the central goal of protocol-to-structure conversion while shifting the project toward a durable, auditable tool that can support trial design and population-comparison applications.

3 Current Web Application

3.1 User-facing workflow

The web interface has three tabs:

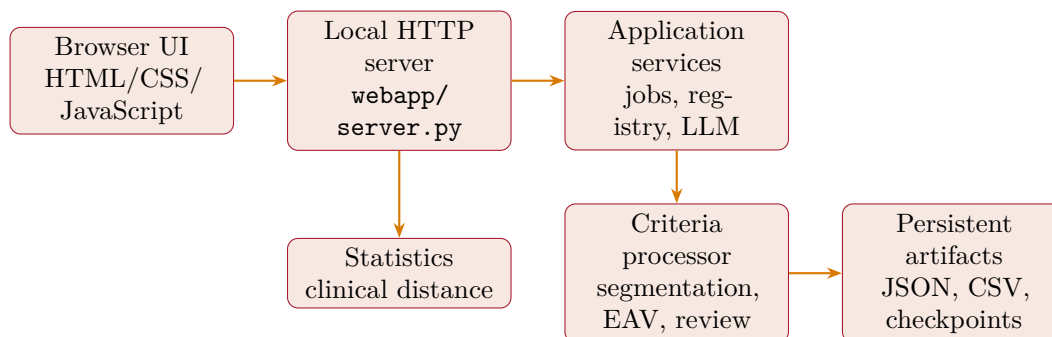
Extract. Enter an NCT ID, paste eligibility text, or do both. When text is blank, the server retrieves the trial from ClinicalTrials.gov. When text is present, it is authoritative and the NCT ID is used as its label. The user can preview deterministic segmentation or run the full AI extraction.

Clinical distance. Compare an observed numerical value with a target criterion using a tolerance, distance scale, and condition weight. The result explains the raw distance, normalized distance, and weighted penalty.

EAV library. Inspect the governed entities and attributes used for extraction, including library revision and content hash.

The criteria text box therefore has two purposes: it permits extraction of a local or unpublished protocol, and it permits an explicit override of the registry text for a supplied NCT ID. Leaving it blank selects registry retrieval.

3.2 Architecture



- `main.py` is the launch point and calls the browser server.
- `webapp/` contains the local server and static browser assets.
- `backend/api/` constructs shared application state.
- `backend/services/` owns registry access, LLM adapters, extraction jobs, costs, and files.
- `backend/criteria_processor/` owns atomic criteria and the governed EAV workflow.
- `backend/ctg_parser/` maps ClinicalTrials.gov v2 JSON into typed trial records.
- `backend/statistics/` owns clinical-distance policies and analysis.
- `config/` contains settings and the governed library; `runtime/eligcrit/` contains recoverable run artifacts.

3.3 End-to-end extraction sequence

1. The browser posts the trial ID and optional criteria text to `/api/jobs/extract`.
2. `EligCritExtractionService.submit` validates the input. With an empty text box it calls `fetch_study`; otherwise it creates a registry-shaped local source document.
3. The source and initial job record are atomically persisted before a background thread begins.
4. The service computes a cache fingerprint from the criteria text, configured model, and EAV-library hash. An unchanged rerun can reuse its audit and breakdown safely.
5. Deterministic segmentation creates provenance-bearing atomic criteria. The LLM pipeline searches candidate attributes, makes a structured selection, normalizes values, and reviews the result.
6. The completed audit is written to disk. The browser polls the job endpoint and then requests a simplified cleaned result for display.

The cache includes the library hash so a terminology update invalidates old mappings. A failed run is not a valid cache entry and can be rerun. Each submission receives a new job ID even when the computed extraction result is reusable.

4 Function and Component Reference

4.1 Entry point, HTTP server, and state

main.main()

Starts the local browser application.

run_browser_app(host, port, open_browser)

Creates the threaded HTTP server, resumes incomplete jobs, optionally opens a browser, and serves until interrupted.

BrowserApplication.dispatch(method, path, body)

Routes health, library, segmentation, normalization, distance, parser, job, result, and cleaned-result API requests.

BrowserApplication.resume()

Delegates recovery of queued or interrupted extraction jobs.

_handler(application)

Builds the request handler, limits request bodies, converts expected exceptions into HTTP responses, and safely serves static files.

_cleaned_item(item)

Projects a full EAV audit item into the columns used by the browser table.

create_app_state()

Loads settings and constructs shared services such as cost monitoring.

4.2 Registry and trial parsing

fetch_study(nct_id)

Validates the NCT identifier and downloads one study from the ClinicalTrials.gov v2 API.

eligibility_text_from_study(study)

Extracts the eligibility-criteria block and reports a clear error when it is absent.

parse_ctgov_study(study)

Converts registry JSON into typed identification, status, design, intervention, outcome, and eligibility fields.

_module(), _date_value()

Defensively retrieve nested protocol modules and normalized date values.

4.3 Extraction jobs and persistence

EligCritExtractionService.submit()

Creates a durable queued job, retrieves or wraps the source, and launches processing.

get(job_id)

Reads the persisted job record or raises a not-found error.

resume_incomplete()

Restarts jobs left queued or running after an application interruption.

_run(job_id)

Runs segmentation and EAV extraction, records cache hits and paths, captures errors, and always writes the final state.

atomic_write_text/json/jsonl()

Writes through a temporary file and atomic replacement so interruption cannot leave a partially written artifact.

FileManager

Provides project file and directory operations for service code.

4.4 LLM boundary and usage accounting

create_llm_provider(settings, cost_monitor)

Constructs the configured provider without exposing vendor details to the pipeline.

OpenAIProvider

Calls the OpenAI SDK and returns a provider-neutral response containing text and token usage.

StructuredJsonAdapter

Requests schema-constrained output and parses the returned JSON object.

RuntimeRecordingJsonLLMProvider

Records prompts, responses, stage metadata, and failures as recoverable JSONL run evidence.

CostMonitor

Records token counts and estimates cost using the selected model's pricing entry.

4.5 Segmentation, extraction, and EAV governance

split_eligibility_criteria()

Separates inclusion and exclusion sections and splits bullets into atomic criteria.

iter_eligibility_source_items()

Returns typed, ordered criteria with stable source provenance.

eligibility_source_items_from_sections()

Creates the same atomic records from already separated sections.

process_trial()

Creates the eligibility breakdown CSV, using checkpoints to avoid losing completed work.

load_breakdown_points()

Validates and loads atomic points for the EAV stage.

normalize_search_expansion() / retrieve_attribute_candidates()

Normalizes LLM search terms and ranks plausible governed attributes using aliases, text overlap, and schema compatibility.

validate_reasoning_selection()

Rejects selections that violate the output schema or select an “existing” attribute outside the supplied candidate set.

build_eva_audit_item() / extract_one_eva()

Constructs one reviewed EAV record from a criterion and its selected mapping.

extract_trial_eva()

Coordinates extraction for every atomic criterion and returns the trial audit document.

finalize_trial_attribute_id_conflicts()

Detects incompatible new definitions that would otherwise share an attribute ID.

EvaLibraryRepository

Loads governed entities and attributes, computes revision/hash identity, takes snapshots, and applies reviewed changes with concurrency checks.

review_eva_mapping() / review_new_attribute_id()

Runs critic-style checks of semantic fit, candidate selection, and proposed identifiers.

validate_mapping_review() / validate_attribute_id_review()

Enforces review schemas, English explanations, candidate hashes, and allowed decisions.

EligibilityEvaReconciliationJobStore

Persists preview/apply reconciliation jobs and their fingerprints.

migrate_eva_audit_v1/v2() / load_eva_audit()

Upgrades historical audit documents into the current schema.

write_audit_csv() / apply_audit_records_to_workbook()

Exports terminology proposals and applies approved entity, attribute, value, and alias updates.

4.6 Value normalization and clinical distance

normalize_structured_value()

Canonicalizes categorical assertions, ranges, units, inclusivity, and criterion polarity.

canonicalize_intervals()

Sorts and merges equivalent or adjacent numeric intervals.

complement_intervals() / intersect_interval_sets()

Computes excluded complements and combined eligible regions.

aggregate_numeric_constraints()

Combines multiple criteria for one attribute into a single interval representation.

validate_numerical_penalty_policy() / validate_categorical_penalty_policy()

Reject malformed penalty policies before analysis.

default_numerical_penalty()

Builds a default tolerance-and-scale policy from target context.

clinical_distance_penalty()

Calculates the weighted penalty for an observed value relative to a numeric target.

categorical_label_penalty() / categorical_unresolved_penalty()

Scores categorical agreement or explicitly unresolved comparisons.

calculate_condition_penalty_analysis()

Evaluates all supported conditions and returns component and aggregate penalties with diagnostics.

initialize_numerical_penalties()

Uses observed/target context to initialize clinically meaningful numerical policy parameters.

5 Meaning of Application Outputs

5.1 Job output

Field or state	Meaning
<code>job_id</code>	Unique identifier for this submission and its run records.
<code>trial_id</code>	Supplied NCT ID or generated <code>LOCAL-*</code> label.
<code>queued</code>	The durable job exists and is waiting for its worker thread.
<code>running</code>	Segmentation, LLM mapping, normalization, or review is active.
<code>completed</code>	A breakdown CSV and EAV audit are available. It does not mean every proposed mapping has been approved for the governed library.
<code>failed</code>	Processing stopped; the <code>error</code> field is the immediate cause. Prior atomic artifacts remain available for diagnosis.
<code>source_path</code>	Registry-shaped JSON used as the immutable run input.
<code>breakdown_path</code>	CSV of atomic criteria and intermediate breakdown fields.
<code>audit_path</code>	Final detailed EAV audit JSON.
<code>cache_hit=true</code>	An audit with identical text, model, and library hash was reused; no new LLM extraction was necessary.

5.2 Cleaned eligibility table

Column	Interpretation
Type	Whether the source criterion is inclusion or exclusion.
Source criterion	Atomic text from the protocol; this preserves traceability to what the model interpreted.
Entity	High-level governed clinical domain.
Attribute	Human-readable attribute name; the smaller identifier is its stable machine key.
Cleaned value	Normalized categorical assertion or numerical interval, with a unit when applicable.
Confidence	Model-reported certainty, useful for triage but not a calibrated probability or proof of correctness.

Column	Interpretation
Review	Governance action produced by mapping review. <code>CONFIRM_EXISTING</code> means the criterion fits a supplied library attribute. <code>REQUIRE_NEW</code> means no existing candidate was adequate and a new attribute is proposed; a human should review it before library admission. Additional rejection or unresolved states, when present, mean the mapping must not be treated as verified.

A `REQUIRE_NEW` row is not an extraction failure. It is a controlled terminology outcome: the clinical statement was extracted, but the governed library lacks a sufficiently equivalent attribute. Conversely, `CONFIRM_EXISTING` confirms the selected mapping under the automated reviewer; it does not replace clinical validation.

5.3 Clinical-distance output

For a scalar observation x , target set T , full-relevance tolerance τ , distance scale $s > 0$, and weight w , the numerical tool conceptually evaluates

$$p(x, T) = w \min\left(1, \frac{\max(0, d(x, T) - \tau)}{s}\right),$$

where $d(x, T)$ is zero inside the target interval and otherwise is the distance to its nearest boundary. Zero means fully compatible under the policy; one times the weight is the capped maximum penalty. The JSON output includes the observed value, target, tolerance, scale, raw clinical distance, normalized penalty, weighted penalty, and diagnostic explanation. These values quantify a declared policy; they are not treatment-effect estimates or medical advice.

6 Meaning of Errors

Errors are returned in API JSON and, for background extraction, copied into the job's `error` field. The exception class identifies the failure category; the text identifies the violated rule.

Error or message	Meaning and response
Enter an NCT ID or paste eligibility criteria text	Both inputs were blank. Provide either source.
Invalid NCT ID / registry 404	The identifier format is invalid or ClinicalTrials.gov has no matching study. Check the ID.
Registry request or timeout error	The v2 API was unavailable, slow, or returned an unexpected response. Retry after checking network access.
Study has no eligibility criteria	The registry record exists but exposes no criteria text usable by this pipeline. Paste the text manually if available.
Eligibility headings or breakdown points missing	Segmentation could not identify usable inclusion/exclusion content, or the intermediate CSV is incomplete. Inspect the source formatting and preview segmentation.

Error or message	Meaning and response
JSON decode/schema error	An HTTP body or LLM response was not valid JSON of the required shape. Run records identify the failing stage.
Existing Attribute selection must use one of the supplied top candidate <code>attribute_id</code> values	The model claimed an existing mapping but invented or reused an ID outside the candidates retrieved for that specific criterion. This is a guardrail failure, not evidence that the chosen ID is clinically valid. Rerun; if repeated, inspect the recorded prompt/candidates or improve the prompt/model.
Categorical, Sex/Gender, range, interval, or unit validation error	The selected value contradicts its schema, polarity, allowed labels, bounds, or canonical unit. Correct the input/library definition or review the LLM selection.
Stale library revision or SHA-256	The governed library changed after a preview was created. Regenerate the preview before applying it; this prevents overwriting concurrent updates.
Attribute ID conflict/collision	Two incompatible definitions propose the same stable identifier. Reconcile or rename before applying.
Job has no completed audit result	The result endpoint was requested before completion or after failure. Poll job status and inspect <code>error</code> .
Missing API key / authentication error	The process did not load a valid provider credential. Put the key in the documented environment file or process environment and restart the server.
Rate limit, quota, or provider error	The upstream model rejected or could not complete the request. Verify quota/model access and retry; the job will be marked failed rather than silently returning partial output.
Request body is too large	The browser sent more than the server's 5 MB safety limit. Reduce the pasted source.
Not found	The API route, static asset, or job ID does not exist. Verify the URL or job identifier.

The saved source, job record, breakdown, checkpoints, and LLM JSONL records form the diagnostic trail. Retrying the same trial is expected to work when inputs are unchanged; a completed matching cache is reused. A repeated candidate-selection error indicates nondeterministic or invalid model output and should be addressed at the recorded mapping stage rather than by deleting the terminology library.

7 Persistence, Recovery, and Privacy

Runtime artifacts are organized beneath `runtime/eligcrit`: source documents, job records, point breakdowns, checkpoints, audits, provider records, and library snapshots. Atomic replacement protects JSON and JSONL records from partial writes. On startup, jobs left in `queued` or `running` are returned to the queue. The application binds to `127.0.0.1` by default, so it is locally accessible rather than a hosted multi-user system. Protocol text and model records may contain sensitive material; runtime directories and `.env` must remain untracked and access-controlled.

8 Key Milestones

1. Reproduced the protocol-to-criteria concept from AutoCriteria.
2. Removed LangChain and introduced direct, provider-neutral OpenAI SDK integration.
3. Refactored the monolithic workflow into backend services, schemas, processing modules, and tests.
4. Built a standalone application and subsequently replaced its desktop GUI with the current full-width browser application.
5. Added ClinicalTrials.gov v2 input, atomic segmentation, structured normalization, governed EAV retrieval, and critic/reconciliation logic.
6. Added recoverable jobs, atomic persistence, cache fingerprints, audit records, and clinical-distance analysis.

9 Future Work

9.1 Fine-tuned embeddings for “seemingly similar” criteria

Generic sentence embeddings overvalue lexical overlap. For example, `Age >= 18` and `Age <= 18` share nearly every token and may receive high cosine similarity, even though their admissible populations differ sharply. A clinical-criteria representation must distinguish semantic topic similarity from constraint equivalence.

The proposed model will use the current extractor as a structured supervision engine. Each criterion is represented by its clinical concept, comparator, threshold or category, unit, inclusion/exclusion polarity, and normalized interval. Development should include:

- a text encoder fine-tuned with contrastive learning so criteria for the same concept remain discoverable while comparator flips, negation, exclusion polarity, and clinically material threshold changes become hard negatives;
- auxiliary heads for entity/attribute classification, comparator and negation classification, typed value extraction, and relation labels such as equivalent, subset, superset, overlap, disjoint, or unknown;
- interval-aware supervision in which structured population overlap supplies a target distance, rather than assuming that lexical cosine distance is clinically meaningful;
- reviewed `CONFIRM_EXISTING` mappings as positive pairs, reconciled aliases as additional positives, and rejected or corrected mappings as high-value negatives;
- retrieval followed by deterministic constraint comparison or an LLM-assisted adjudicator, with the latter required to return structured evidence rather than an unsupported scalar;
- calibration and abstention so ambiguous criteria are surfaced for review instead of forced into misleading proximity.

A practical loss can combine contrastive embedding loss with operator, polarity, value, and relation losses. Evaluation must be split by clinical attribute to limit leakage and must report retrieval recall, ranking quality, relation accuracy, calibration, and threshold sensitivity. A mandatory challenge set should include `Age >= 18` versus `Age <= 18`, nearby age thresholds, unit conversions, negation, and inclusion/exclusion reversals. The model should not be released if it ranks a comparator-flipped criterion as equivalent merely because the sentences are lexically similar.

The resulting representation can support duplicate detection, terminology suggestions, and clustering. More importantly, its constraint-aware distance can propagate into trial design: quantify how far a proposed eligibility rule moves from a reference population, simulate the resulting coverage and recruitment changes, and examine whether relaxing or tightening a rule changes power, sample

size, or expected generalizability.

9.2 Broader roadmap

Additional work includes terminology links to RxNorm and ICD-10 for automatic cleaning, a separately evaluated reviewer/critic agent, adaptive dictionary updates based on embeddings and governed human approval, multi-user hosted deployment with authentication, and downstream protocol-to-design and protocol-to-analysis agents. The long-term goal is an agentic clinical-trial system that extracts eligibility constraints, validates them, compares populations, simulates trial consequences, and produces auditable statistical design artifacts while keeping clinicians and study coordinators in the approval loop.

10 Conclusion

The current project is no longer a single-script extraction demonstration. It is a browser-delivered, layered application with registry ingestion, atomic segmentation, structured LLM extraction, EAV governance, reviewer logic, value normalization, clinical-distance analysis, persistence, recovery, and explicit audit outputs. Its next technical challenge is to turn structured criteria into a constraint-aware population representation: one that recognizes shared clinical concepts without confusing opposite or materially different eligibility rules.

References

- [1] S. Datta, *AutoCriteria*, GitHub repository.
<https://github.com/surdatta/autocriteria>
- [2] Y. Zhao, *EligCrit*, GitHub repository.
<https://github.com/JackZhao1998/EligCrit>
- [3] S. Datta et al., *AutoCriteria: A Generalizable Clinical Trial Eligibility Criteria Extraction System Powered by Large Language Models*, 2023.
<https://arxiv.org/abs/2304.06096>